

**ОПИСАНИЕ ПРОГРАММНЫХ МОДУЛЕЙ
СИСТЕМЫ ПЛАНИРОВАНИЯ
ПРОИЗВОДСТВА (APS)
НА ОСНОВЕ ЦИФРОВОГО ДВОЙНИКА
ГАЛЬВАНИЧЕСКОГО КОМПЛЕКСА**

Приложение к диссертационной работе

Содержание

1	Введение	3
2	Архитектура системы и протоколы взаимодействия	4
3	Серверное ядро (Java / Spring Boot)	5
3.1	SmartFactoryApplication.java — Точка входа приложения . .	5
3.2	ApiState.java — Глобальное состояние системы и 7 стадий . .	5
3.3	BatchInfo.java — Модель производственной партии	8
3.4	ComplexParameters.java — Параметры оборудования и расчёт загрузки	9
3.5	SuperpositionData.java — Файл суперпозиции расписания . .	9
3.6	ApsAlgorithmService.java — Реализация 12 блоков алгоритма APS	10
3.6.1	Математическая модель приоритетов	10
3.6.2	Расчёт времени обработки партии	10
3.7	SimulationService.java — Симуляция стадий и запуск APS-цикла	11
3.8	ScheduleService.java — Потокбезопасная работа с расписанием	12
3.9	FactoryController.java — REST API контроллер	13
3.10	OpcUaServerManager.java — Менеджер OPC UA сервера . .	14
3.11	FactoryNamespace.java — Адресное пространство OPC UA .	14
4	Аппаратный мост и сбор телеметрии (Python / FastAPI / UART)	17
4.1	hardware_bridge.py — Аппаратно-программный шлюз HIL .	17
4.1.1	Протокол обмена по UART	17
4.1.2	Ретрансляция в Java-сервер	17
5	Визуальный 2D-симулятор (Python / PyGame)	19
5.1	main.py — Графический SCADA-клиент OPC UA	19
5.1.1	Двухъярусная топология линии	19
5.1.2	Автомат состояний мостового манипулятора	19
6	Единый диспетчер запуска (C# / .NET / Release Launcher)	21
6.1	Launcher.cs — Оркестратор процессов и мониторинг здоровья	21
6.1.1	Функции оркестратора	21
7	Веб-дашборд (React / TypeScript / Vite)	23
7.1	App.tsx — SCADA-панель цифрового двойника	23
7.1.1	Функциональные блоки интерфейса	23
8	Мобильный терминал (Android / Kotlin)	24
8.1	Архитектура мобильного клиента	24

9	Вспомогательные файлы и конфигурации	25
9.1	schedule.json — Персистентное расписание	25
10	Заключение	26

1 Введение

Настоящий документ содержит подробное описание каждого программного модуля (скрипта и класса), входящего в состав распределённой системы автоматизированного динамического планирования производства (APS — Advanced Planning and Scheduling) для технологического комплекса горячего цинкования.

Система представляет собой цифровой двойник промышленной гальванической линии, состоящей из 7 последовательных технологических стадий, и включает следующие взаимосвязанные подсистемы:

1. **Серверное ядро** (Spring Boot, Java 17+) — центральный узел вычислений алгоритма APS, встроенный сервер промышленного протокола OPC UA (на базе Eclipse Milo), HTTP REST API для веб- и мобильных клиентов;
2. **Аппаратный мост и телеметрия** (FastAPI, Python 3.10+, PySerial) — аппаратно-программный шлюз Hardware-in-the-Loop (HIL), выполняющий чтение телеметрии физического микроконтроллера ESP32 (датчики DHT11/DHT22) по последовательному интерфейсу UART (COM-порт 115200 бод), локальную симуляцию смежных стадий и ретрансляцию данных в Java-сервер;
3. **Визуальный 2D-симулятор** (PyGame, Python, AsyncUA) — графический SCADA-клиент OPC UA, визуализирующий в реальном времени работу многопозиционной технологической линии, физику перемещения манипулятора (мостового крана) и динамику технологических параметров ванн;
4. **Единый оркестратор запуска** (Launcher, C# / .NET) — системная утилита координации жизненного цикла процессов, проверки доступности портов, автоматического запуска веб-браузера и безопасного аварийного завершения дочерних процессов;
5. **Веб-дашборд** (React 18 / Vite, TypeScript, Tailwind CSS) — интерактивная панель мониторинга цифрового двойника, отображающая ход выполнения блок-схемы алгоритма APS, телеметрию всех 7 технологических стадий, очереди MES и системные логи;
6. **Мобильный терминал** (Android, Kotlin / Jetpack Compose) — мобильный SCADA-терминал оператора линии для оперативного удалённого мониторинга и ручной корректировки очереди.

Все программные модули описаны в строгой технической форме с приведением архитектурных особенностей, алгоритмических формул и ключевых фрагментов исходного кода (листингов).

2 Архитектура системы и протоколы взаимодействия

Система построена на принципах сервис-ориентированной распределённой архитектуры. Обмен данными между компонентами осуществляется по четырём специализированным каналам связи:

- **UART / Serial** (COM20, 115200 бод, 8N1) — физический канал связи с контроллером ESP32 для сбора первичных показаний температуры и влажности сушильной камеры;
- **OPC UA** (TCP-порт 4840) — бинарный промышленный протокол связи реального времени (спецификация IEC 62541). Обеспечивает циклическую передачу телеметрии и команд управления между Java-сервером и PyGame-симулятором с частотой 5–10 Гц;
- **REST API Java-сервера** (HTTP-порт 8080) — интерфейс интеграции веб-дашборда, Android-приложения и аппаратного моста. Поддерживает поллинг с частотой 2 Гц;
- **FastAPI Аппаратного моста** (HTTP-порт 8000) — локальный REST-сервер для автономного мониторинга телеметрии датчиков и ретрансляции данных.

Таблица 1: Пакетная структура серверного ядра (Java)

Пакет	Назначение
<code>com.smartfactory</code>	Точка входа в приложение и конфигурация планировщика задач
<code>com.smartfactory.model</code>	Модели данных (партии <code>BatchInfo</code> , параметры комплекса <code>ComplexParameters</code> , суперпозиция <code>SuperpositionData</code> , действия <code>ActionSequence</code>)
<code>com.smartfactory.state</code>	Глобальное состояние системы и 7 технологических стадий (<code>ApiState</code>)
<code>com.smartfactory.service</code>	Бизнес-логика алгоритма APS, управление расписанием, симуляция процессов
<code>com.smartfactory.controller</code>	Контроллер REST API для клиентов и аппаратного моста
<code>com.smartfactory.opcua</code>	Встроенный OPC UA сервер и адресное пространство узлов

3 Серверное ядро (Java / Spring Boot)

3.1 SmartFactoryApplication.java — Точка входа приложения

Путь: com.smartfactory.SmartFactoryApplication

Зависимости: Spring Boot 3.x, Spring Scheduling, Eclipse Milo

Класс инициализирует контекст Spring Boot и настраивает пул потоков планировщика задач. По умолчанию стандартный планировщик Spring Boot использует один поток, что создаёт риск взаимных блокировок между высокочастотным опросом OPC UA, вычислениями алгоритма APS и симуляцией датчиков.

Для исключения взаимного влияния настраивается специализированный бин TaskScheduler с фиксированным пулом из 5 независимых потоков (aps-scheduler-).

```
1 package com.smartfactory;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.context.annotation.Bean;
6 import org.springframework.scheduling.TaskScheduler;
7 import org.springframework.scheduling.annotation.EnableScheduling;
8 import org.springframework.scheduling.concurrent.ThreadPoolTaskScheduler;
9
10 @SpringBootApplication
11 @EnableScheduling
12 public class SmartFactoryApplication {
13
14     public static void main(String[] args) {
15         SpringApplication.run(SmartFactoryApplication.class, args);
16     }
17
18     @Bean
19     public TaskScheduler taskScheduler() {
20         ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
21         scheduler.setPoolSize(5);
22         scheduler.setThreadNamePrefix("aps-scheduler-");
23         scheduler.initialize();
24         return scheduler;
25     }
26 }
```

Листинг 1: Фрагмент SmartFactoryApplication.java

3.2 ApiState.java — Глобальное состояние системы и 7 стадий

Путь: com.smartfactory.state.ApiState

Аннотации: @Component, @Data (Lombok)

Класс реализует потокобезопасный синглтон, хранящий полный слепок состояния цифрового двойника. Помимо исходных общесистемных флагов, класс расширен полями состояния 7 технологических стадий горячего цинкования:

1. **Стадия 1 (Stage1Degreasing):** Щелочное обезжиривание ($T \approx 50^{\circ}\text{C}$, флаг активности, управление нагревателем ТЭН);
2. **Стадия 2 (Stage2Rinsing):** Промывка №1 (каскадный циркуляционный насос);
3. **Стадия 3 (Stage3Pickling):** Кислотное травление в соляной кислоте ($T \approx 25^{\circ}\text{C}$, перемешивающее устройство/барботажа);
4. **Стадия 4 (Stage4Rinsing):** Промывка №2 (промывочный насос);
5. **Стадия 5 (Stage5Fluxing):** Флюсование ($T \approx 60^{\circ}\text{C}$, нагреватель);
6. **Стадия 6 (Stage6DryingChamber):** Сушильная камера (температура, влажность от физического датчика через UART, вентилятор обдува, индикатор подключения датчика);
7. **Стадия 7 (Stage7ZincBath):** Ванна горячего цинкования ($T \approx 450^{\circ}\text{C}$, расплав цинка, электронагрев).

Метод `getFactoryStateMap()` формирует структуру данных, идентичную JSON-схеме аппаратного моста на Python.

```
1 package com.smartfactory.state;
2
3 import lombok.Data;
4 import org.springframework.stereotype.Component;
5 import java.util.*;
6
7 @Component
8 @Data
9 public class ApiState {
10     private int temperature = 0;
11     private int humidity = 0;
12     private boolean doorClosed = true;
13     private boolean manualMode = false;
14     private String status = "STARTING";
15     private String step = "Инициализация";
16     private boolean simRunning = false;
17     private boolean clientConnected = false;
18
19     // Метрики алгоритма APS
20     private String apsPhase = "IDLE";
21     private double utilizationCoeff = 0.0;
22     private int processingTimeEstimate = 0;
23     private boolean canIncludeResult = false;
24     private String currentBatch = "";
25     private String rejectionCause = "";
26     private int recalculationCount = 0;
27     private int totalCyclesCompleted = 0;
28
29     // Состояние 7 технологических узлов линии цинкования
30     private Stage1Degreasing stage1 = new Stage1Degreasing();
31     private Stage2Rinsing stage2 = new Stage2Rinsing();
32     private Stage3Pickling stage3 = new Stage3Pickling();
33     private Stage4Rinsing stage4 = new Stage4Rinsing();
```

```

34     private Stage5Fluxing stage5 = new Stage5Fluxing();
35     private Stage6DryingChamber stage6 = new Stage6DryingChamber();
36     private Stage7ZincBath stage7 = new Stage7ZincBath();
37
38     private List<String> apsHistory = new ArrayList<>();
39     private List<LogEntry> systemLog = new ArrayList<>();
40
41     public Map<String, Object> getFactoryStateMap() {
42         return Map.of(
43             "stage_1_alkaline_degreasing", Map.of("temperature", stage1.
44 getTemperature(), "is_active", stage1.isActive(), "heater_on", stage1.
45 isHeaterOn()),
46             "stage_2_rinsing_1", Map.of("is_active", stage2.isActive(), "
47 pump_running", stage2.isPumpRunning()),
48             "stage_3_pickling", Map.of("temperature", stage3.
49 getTemperature(), "is_active", stage3.isActive(), "agitator_on", stage3.
50 isAgitatorOn()),
51             "stage_4_rinsing_2", Map.of("is_active", stage4.isActive(), "
52 pump_running", stage4.isPumpRunning()),
53             "stage_5_fluxing", Map.of("temperature", stage5.getTemperature
54 (), "is_active", stage5.isActive(), "heater_on", stage5.isHeaterOn()),
55             "stage_6_drying_chamber", Map.of("temperature_celsius", stage6
56 .getTemperatureCelsius(), "humidity_percent", stage6.getHumidityPercent
57 (), "is_active", stage6.isActive(), "fan_on", stage6.isFanOn(), "
58 sensor_connected", stage6.isSensorConnected()),
59             "stage_7_zinc_bath", Map.of("temperature", stage7.
60 getTemperature(), "is_active", stage7.isActive(), "heater_on", stage7.
61 isHeaterOn())
62         );
63     }
64
65     public synchronized void addApsHistoryEntry(String entry) {
66         apsHistory.add(entry);
67         if (apsHistory.size() > 20) apsHistory = new ArrayList<>(
68 apsHistory.subList(apsHistory.size() - 20, apsHistory.size()));
69     }
70
71     public synchronized void addSystemLog(String source, String message) {
72         String time = java.time.LocalDateTime.now().format(java.time.format.
73 DateTimeFormatter.ofPattern("HH:mm:ss"));
74         systemLog.add(new LogEntry(time, source, message));
75         if (systemLog.size() > 30) systemLog = new ArrayList<>(systemLog.
76 subList(systemLog.size() - 30, systemLog.size()));
77     }
78
79     @Data public static class Stage1Degreasing { private int temperature =
80 50; private boolean isActive = false; private boolean heaterOn = false
81 ; }
82     @Data public static class Stage2Rinsing { private boolean isActive =
83 false; private boolean pumpRunning = false; }
84     @Data public static class Stage3Pickling { private int temperature =
85 25; private boolean isActive = false; private boolean agitatorOn =
86 false; }
87     @Data public static class Stage4Rinsing { private boolean isActive =
88 false; private boolean pumpRunning = false; }
89     @Data public static class Stage5Fluxing { private int temperature =
90 60; private boolean isActive = false; private boolean heaterOn = false;
91 }
92     @Data public static class Stage6DryingChamber { private int
93 temperatureCelsius = 55; private int humidityPercent = 20; private

```

```

boolean isActive = false; private boolean fanOn = false; private
boolean sensorConnected = true; }
70 @Data public static class Stage7ZincBath { private int temperature =
450; private boolean isActive = true; private boolean heaterOn = false;
}
71 @Data @lombok.AllArgsConstructor @lombok.NoArgsConstructor
72 public static class LogEntry { private String time; private String
source; private String message; }
73 }

```

Листинг 2: Структура ApiState.java и описание 7 стадий

3.3 BatchInfo.java — Модель производственной партии

Путь: `com.smartfactory.model.BatchInfo`

Класс описывает партию деталей, включая её идентификатор, наименование, физические параметры (массу, материал) и вычисленный алгоритмом APS динамический приоритет P . Содержит фабричный метод `fromLegacyName` для автоматического синтаксического разбора текстовых заказов оператора с выделением веса и технологического типа сырья.

```

1 package com.smartfactory.model;
2
3 import lombok.Data;
4 import lombok.NoArgsConstructor;
5 import java.util.ArrayList;
6 import java.util.List;
7
8 @Data
9 @NoArgsConstructor
10 public class BatchInfo {
11     private String id;
12     private String name;
13     private String material = "Сталь";
14     private double weight = 50.0;
15     private int priority = 0;
16     private String assignedLineId = "LINE_1";
17     private int estimatedTimeSec = 0;
18     private BatchStatus status = BatchStatus.WAITING;
19     private List<ProcessingStep> steps = new ArrayList<>();
20
21     public enum BatchStatus { WAITING, PRIORITIZED, SCHEDULED, ACTIVE,
COMPLETED, REJECTED }
22
23     @Data
24     public static class ProcessingStep {
25         private int stepNumber;
26         private String equipmentId;
27         private String operationName;
28         private int durationSec;
29     }
30
31     public static BatchInfo fromLegacyName(String name) {
32         BatchInfo b = new BatchInfo();
33         b.setId("BATCH_" + System.currentTimeMillis() % 100000);
34         b.setName(name);
35         if (name.contains("Медь")) b.setMaterial("Медь");

```

```

36         else if (name.contains("Алюм")) b.setMaterial("Алюминий");
37         else b.setMaterial("Сталь");
38         return b;
39     }
40 }

```

Листинг 3: Фрагмент модели BatchInfo.java

3.4 ComplexParameters.java — Параметры оборудования и расчёт загрузки

Путь: com.smartfactory.model.ComplexParameters

Инкапсулирует конфигурацию промышленного комплекса, состав производственных линий и перечень единиц оборудования. Реализует математическую модель оценки средней загруженности оборудования U_{avg} :

$$U_{avg} = \frac{1}{N} \sum_{i=1}^N \frac{L_i}{C_i} \quad (1)$$

где L_i — текущая фактическая загрузка i -го агрегата (кг), C_i — паспортная ёмкость (кг), N — число задействованных агрегатов с ненулевой ёмкостью. Порог критической перегрузки установлен на уровне $U_{crit} = 0,85$ (85%).

```

1 public double getAverageUtilization() {
2     double total = 0.0;
3     int count = 0;
4     for (Equipment eq : equipmentMap.values()) {
5         if (eq.getCapacityKg() > 0 && eq.isAvailable()) {
6             total += (double) eq.getCurrentLoadKg() / eq.getCapacityKg();
7             count++;
8         }
9     }
10    return count > 0 ? total / count : 0.0;
11 }

```

Листинг 4: Расчёт коэффициента загруженности в ComplexParameters.java

3.5 SuperpositionData.java — Файл суперпозиции расписания

Путь: com.smartfactory.model.SuperpositionData

Реализует концепт суперпозиции: наложение расчётного времени технологических операций партии на временную сетку загрузки оборудования с учётом свободных технологических окон.

```

1 @Data
2 public static class TimeSlot {
3     private String equipmentId;
4     private int startTimeSec;

```

```

5     private int endTimeSec;
6     private boolean occupied;
7     private String batchId;
8 }
9
10 @Data
11 public static class FitResult {
12     private boolean canFit;
13     private String rejectionReason;
14     private int proposedStartSec;
15     private int proposedEndSec;
16     private double requiredCapacity;
17     private double availableCapacity;
18 }

```

Листинг 5: Классы тайм-слота и результата сопоставления в SuperpositionData.java

3.6 ApsAlgorithmService.java — Реализация 12 блоков алгоритма APS

Путь: com.smartfactory.service.ApsAlgorithmService

Размер: 718 строк

Зависимости: ApiState, ScheduleService

Является центральным вычислительным ядром системы. Каждый публичный метод строго соответствует функциональному блоку теоретической блок-схемы алгоритма APS.

3.6.1 Математическая модель приоритетов

Приоритет партии P вычисляется как композиция срочности ожидания U , коэффициента сырья M и массового фактора W :

$$P = U + M + W + P_{manual} \quad (2)$$

где $U = \min(50, t_{wait}/10)$, $M \in \{30, 20, 10\}$ (для меди, алюминия и стали соответственно), $W = \min(20, m/5)$. При ручном вклинивании оператора задаётся абсолютный приоритет $P_{manual} = 999$.

3.6.2 Расчёт времени обработки партии

Расчётное время T_{proc} учитывает технологическую длительность каждой операции на назначенном оборудовании, скорректированную на температурно-материальный коэффициент K_m и относительную массу заготовки w :

$$T_{proc} = \sum_{k=1}^S T_{base,k} \cdot K_m \cdot \left(0,8 + 0,4 \cdot \frac{w}{w_{max}} \right) \quad (3)$$

```

1 public List<BatchInfo> determineBatchPriority(List<BatchInfo> queue) {
2     long now = System.currentTimeMillis();
3     for (BatchInfo batch : queue) {
4         int urgencyScore = (int) Math.min(50, (now - batch.getCreatedAt())
5         / 10000);
6         int materialScore = switch (batch.getMaterial()) {
7             case "Медь" -> 30;
8             case "Алюминий" -> 20;
9             default -> 10;
10        };
11        int weightScore = (int) Math.min(20, batch.getWeight() / 5);
12        batch.setPriority(urgencyScore + materialScore + weightScore);
13    }
14    queue.sort((a, b) -> Integer.compare(b.getPriority(), a.getPriority()));
15    return queue;
16 }
17 public SuperpositionData.FitResult compareWithRealSelection(BatchInfo
18     batch, SuperpositionData supData) {
19     SuperpositionData.FitResult result = new SuperpositionData.FitResult()
20     ;
21     double util = complexParams.getAverageUtilization();
22     if (util >= 0.85) {
23         result.setCanFit(false);
24         result.setRejectionReason("Превышен порог загруженности оборудован
25     ия (" + String.format("%.1f%%", util * 100) + " >= 85%)");
26         return result;
27     }
28     if (batch.getEstimatedTimeSec() > 3600) {
29         result.setCanFit(false);
30         result.setRejectionReason("Расчётное время обработки превышает доп
31     устимый интервал смены");
32         return result;
33     }
34     result.setCanFit(true);
35     return result;
36 }

```

Листинг 6: Фрагменты вычисления приоритета и времени в ApsAlgorithmService.java

3.7 SimulationService.java — Симуляция стадий и запуск APS-цикла

Путь: `com.smartfactory.service.SimulationService`

Класс объединяет три циклических планируемых процесса:

1. `sensorUpdateLoop()` (каждую 1 секунду) — сбор и синхронизация показаний датчиков сушильной камеры (DHT) и контроль концефика защитного ограждения;
2. `galvanicStagesSimulationLoop()` (каждые 2 секунды) — математическая симуляция термогидравлических параметров 7 технологических стадий (гистерезис нагревателей, работа циркуляционных насо-

сов, перемешивание барботером, подогрев флюса, расплав цинка при 450°C);

3. `globalSchedulerLoop()` (каждые 8 секунд) — последовательное пошаговое выполнение полного цикла блок-схемы APS из 12 этапов с записью результатов в журнал истории.

```
1 @Scheduled(fixedRate = 2000)
2 public void galvanicStagesSimulationLoop() {
3     if (!apiState.isSimRunning()) return;
4
5     // Узел 1: Щелочное обезжиривание (50-60 C)
6     int t1 = 45 + random.nextInt(11);
7     apiState.getStage1().setTemperature(t1);
8     apiState.getStage1().setHeaterOn(t1 < 50);
9     apiState.getStage1().setActive(random.nextBoolean());
10
11    // Узел 2: Промывка 1 (насос)
12    boolean pump2 = random.nextBoolean();
13    apiState.getStage2().setPumpRunning(pump2);
14    apiState.getStage2().setActive(pump2);
15
16    // Узел 3: Травление (20-30 C, мешалка)
17    int t3 = 20 + random.nextInt(11);
18    apiState.getStage3().setTemperature(t3);
19    apiState.getStage3().setAgitatorOn(random.nextBoolean());
20    apiState.getStage3().setActive(random.nextBoolean());
21
22    // Узел 4: Промывка 2
23    boolean pump4 = random.nextBoolean();
24    apiState.getStage4().setPumpRunning(pump4);
25    apiState.getStage4().setActive(pump4);
26
27    // Узел 5: Флюс (55-65 C)
28    int t5 = 55 + random.nextInt(11);
29    apiState.getStage5().setTemperature(t5);
30    apiState.getStage5().setHeaterOn(t5 < 60);
31    apiState.getStage5().setActive(random.nextBoolean());
32
33    // Узел 7: Ванна цинкования (440-460 C)
34    int t7 = 440 + random.nextInt(21);
35    apiState.getStage7().setTemperature(t7);
36    apiState.getStage7().setHeaterOn(t7 < 448);
37    apiState.getStage7().setActive(true);
38 }
```

Листинг 7: Фрагмент `SimulationService.java` с симуляцией стадий цинкования

3.8 `ScheduleService.java` — Потокбезопасная работа с расписанием

Путь: `com.smartfactory.service.ScheduleService`

Обеспечивает транзакционное чтение и модификацию файла `schedule.json`. Защита от состояния гонки между планировщиком APS и внешними REST-запросами клиентов реализована через монитор синхронизации `fileLock`.

```

1 @Service
2 public class ScheduleService {
3     private final ObjectMapper mapper = new ObjectMapper();
4     private final File scheduleFile = new File("schedule.json");
5     private final Object fileLock = new Object();
6
7     public ScheduleData readSchedule() {
8         synchronized (fileLock) {
9             if (!scheduleFile.exists()) return new ScheduleData();
10            try {
11                return mapper.readValue(scheduleFile, ScheduleData.class);
12            } catch (IOException e) {
13                return new ScheduleData();
14            }
15        }
16    }
17
18    public void writeSchedule(ScheduleData data) {
19        synchronized (fileLock) {
20            try {
21                mapper.writerWithDefaultPrettyPrinter().writeValue(
22                scheduleFile, data);
23            } catch (IOException e) {
24                e.printStackTrace();
25            }
26        }
27    }
28 }

```

Листинг 8: Фрагмент ScheduleService.java

3.9 FactoryController.java — REST API контроллер

Путь: com.smartfactory.controller.FactoryController

Базовый URL: /api

Предоставляет кросс-доменный HTTP-интерфейс (CORS) для веб-дашборда, мобильного терминала, оркестратора Launcher и Python-моста. Включает специализированные эндпоинты /api/factory-status (двунаправленный обмен телеметрией стадий) и /api/health (liveness-проба готовности ядра).

```

1 @RestController
2 @CrossOrigin(origins = "*")
3 public class FactoryController {
4     private final ApiState apiState;
5     private final ScheduleService scheduleService;
6
7     @GetMapping("/api/factory-status")
8     public Map<String, Object> getFactoryStatus() {
9         return Map.of("status", "ok", "data", apiState.getFactoryStateMap(
10         ));
11     }
12
13     @PostMapping("/api/factory-status")
14     public Map<String, Object> updateFactoryStatus(@RequestBody Map<String,
15     Object> payload) {
16
17     }
18 }

```

```

14      // Приём телеметрии от Python аппаратного моста
15      return Map.of("status", "ok");
16  }
17
18  @GetMapping({"/api/health", "/health"})
19  public Map<String, Object> healthCheck() {
20      return Map.of(
21          "status", "UP",
22          "opcua_server", "RUNNING",
23          "sim_running", apiState.isSimRunning(),
24          "manual_mode", apiState.isManualMode()
25      );
26  }
27  }

```

Листинг 9: Эндпоинты FactoryController.java

3.10 OpcUaServerManager.java — Менеджер OPC UA сервера

Путь: com.smartfactory.opcua.OpcUaServerManager

Выполняет жизненный цикл сервера Eclipse Milo на порту 4840. При запуске регистрирует конечную точку `opc.tcp://0.0.0.0:4840/`, подключает пространство имён `FactoryNamespace` и запускает периодический цикл синхронизации (каждые 200 мс).

```

1  @Scheduled(fixedRate = 200)
2  public void syncWithOpcUa() {
3      if (factoryNamespace == null) return;
4      boolean active = factoryNamespace.isClientActive();
5      apiState.setClientConnected(active);
6      if (!active) return;
7
8      // Считывание команд PyGame-симулятора
9      factoryNamespace.readClientValuesToApiState(apiState);
10     // Трансляция статусов APS и параметров стадий в узлы OPC UA
11     factoryNamespace.writeServerValuesToOpc(apiState);
12 }

```

Листинг 10: Синхронизация переменных в OpcUaServerManager.java

3.11 FactoryNamespace.java — Адресное пространство OPC UA

Путь: com.smartfactory.opcua.FactoryNamespace

URI пространства: `http://factory-aps.local`

Создаёт и организует иерархическую структуру узлов стандарта IEC 62541. Содержит 8 тематических папок-узлов, полностью покрывающих физическую модель линии цинкования:

Таблица 2: Структура адресного пространства
OPC UA

Узел (Папка)	Переменная	Тип	Назначение
1_Alkaline_Degreasing	Temperature	Int32	Температура щелочного раствора (°C)
	IsActive	Boolean	Флаг нахождения детали в ванне
	HeaterOn	Boolean	Состояние нагрева ТЭН
2_Rinsing_1	IsActive	Boolean	Флаг промывки детали
	PumpRunning	Boolean	Работа циркуляционного насоса
3_Pickling	Temperature	Int32	Температура ванны травления (°C)
	IsActive	Boolean	Флаг травления детали в кислоте
	AgitatorOn	Boolean	Состояние перемешивателя
4_Rinsing_2	IsActive	Boolean	Флаг финишной промывки
	PumpRunning	Boolean	Работа насоса промывки №2
5_Fluxing	Temperature	Int32	Температура растопки флюсов (°C)
	IsActive	Boolean	Флаг пассивации ванны
	HeaterOn	Boolean	Подогрев ванны флюсом
6_Drying_Chamber	TemperatureCelsius	Int32	Температура сушильной камеры UART
	HumidityPercent	Int32	Относительная влажность по датчику
	IsActive	Boolean	Процесс сушки заготовки
	FanOn	Boolean	Включение вентилятора обдува
7_Zinc_Bath	SensorConnected	Boolean	Статус связи с датчиком UART
	Temperature	Int32	Температура расплава цинка (450°C)
	IsActive	Boolean	Погружение заготовки в расплав
8_Global_Scheduler	HeaterOn	Boolean	Мощный электроподогрев ванны
	SimulationRunning	Boolean	Пуск/останов симуляции

Узел (Папка)	Переменная	Тип	Назначение
	ManualOverride	Boolean	Флаг ручного вмешательства (кнопка G)
	SystemStatus	String	Текущий глобальный статус APS
	CurrentAlgorithmStep	String	Название активной блок-схемы

```

1 public void registerNodes() {
2     UByte rwAccess = UByte.valueOf(3); // Чтение и запись
3
4     UaFolderNode folder1 = createFolder("1_Alkaline_Degreasing");
5     stage1TempNode = createVariable(folder1.getNodeId(), "1
6     _Alkaline_Degreasing/Temperature", "Temperature", Identifiers.Int32,
7     new Variant(50), rwAccess);
8     stage1ActiveNode = createVariable(folder1.getNodeId(), "1
9     _Alkaline_Degreasing/IsActive", "IsActive", Identifiers.Boolean, new
10    Variant(false), rwAccess);
11    stage1HeaterNode = createVariable(folder1.getNodeId(), "1
12    _Alkaline_Degreasing/HeaterOn", "HeaterOn", Identifiers.Boolean, new
13    Variant(false), rwAccess);
14
15    // Аналогично регистрируются узлы 2..7
16
17    UaFolderNode sched = createFolder("8_Global_Scheduler");
18    simRunningNode = createVariable(sched.getNodeId(), "
19    SimulationRunning", "SimulationRunning", Identifiers.Boolean, new
20    Variant(false), rwAccess);
21    manualOverrideNode = createVariable(sched.getNodeId(), "ManualOverride
22    ", "ManualOverride", Identifiers.Boolean, new Variant(false), rwAccess)
23    ;
24    systemStatusNode = createVariable(sched.getNodeId(), "SystemStatus",
25    "SystemStatus", Identifiers.String, new Variant("WAITING"), rwAccess);
26    currentStepNode = createVariable(sched.getNodeId(), "
27    CurrentAlgorithmStep", "CurrentAlgorithmStep", Identifiers.String, new
28    Variant("Ожидание"), rwAccess);
29 }
30
31 public void writeServerValuesToOpc(ApiState state) {
32     if (systemStatusNode != null) systemStatusNode.setValue(new DataValue(
33     new Variant(state.getStatus())));
34     if (currentStepNode != null) currentStepNode.setValue(new DataValue(
35     new Variant(state.getStep())));
36     if (stage1TempNode != null) stage1TempNode.setValue(new DataValue(new
37     Variant(state.getStage1().getTemperature())));
38     if (stage7TempNode != null) stage7TempNode.setValue(new DataValue(new
39     Variant(state.getStage7().getTemperature())));
40     // Обновление остальных 6 узлов
41 }

```

Листинг 11: Регистрация узлов и запись телеметрии в FactoryNamespace.java

4 Аппаратный мост и сбор телеметрии (Python / FastAPI / UART)

4.1 hardware_bridge.py — Аппаратно-программный шлюз HIL

Путь: hardware_bridge.py

Язык: Python 3.10+

Зависимости: FastAPI, Uvicorn, PySerial, AsyncIO

Модуль обеспечивает физический уровень сбора данных (Edge/Fog). Выполняет непрерывный неблокирующий опрос последовательного COM-порта (COM20, 115200 бод), к которому подключён микроконтроллер ESP32 с цифровым датчиком DHT11/DHT22.

4.1.1 Протокол обмена по UART

ESP32 передаёт пакеты в текстовом формате ASCII, разделённые точкой с запятой:

```
temp:24;hum:48\n
```

Функция `read_serial_data()` парсит значения температуры и относительной влажности, обновляет параметры узла 6 (`stage_6_drying_chamber`) и переключает признак подключения физического датчика (`sensor_connected = True`). При обрыве соединения или отключении кабеля модуль фиксирует ошибку, переводит флаг в `False` и переключается на встроенный программный генератор.

4.1.2 Ретрансляция в Java-сервер

Фоновая корутина `forward_to_java_server()` с периодичностью 1 раз в секунду выполняет HTTP POST-запрос на адрес `http://localhost:8080/api/fa` передавая текущее состояние всех узлов в JSON-формате.

```
1 import asyncio, random, urllib.request, json
2 from contextlib import asynccontextmanager
3 from fastapi import FastAPI
4 from fastapi.middleware.cors import CORSMiddleware
5 try:
6     import serial
7 except ImportError:
8     serial = None
9
10 COM_PORT = 'COM20'
11 BAUD_RATE = 115200
12
13 factory_state = {
14     "stage_1_alkaline_degreasing": {"temperature": 50, "is_active": False,
15     "heater_on": False},
```

```

15     "stage_2_rinsing_1": {"is_active": False, "pump_running": False},
16     "stage_3_pickling": {"temperature": 25, "is_active": False, "
agitator_on": False},
17     "stage_4_rinsing_2": {"is_active": False, "pump_running": False},
18     "stage_5_fluxing": {"temperature": 60, "is_active": False, "heater_on"
: False},
19     "stage_6_drying_chamber": {"temperature_celsius": 0, "humidity_percent
": 0, "is_active": False, "fan_on": False, "sensor_connected": False},
20     "stage_7_zinc_bath": {"temperature": 450, "is_active": True, "
heater_on": False}
21 }
22
23 ser = None
24
25 async def read_serial_data():
26     global factory_state, ser
27     while True:
28         try:
29             if ser and ser.is_open and ser.in_waiting > 0:
30                 line = ser.readline().decode('utf-8', errors='ignore').
strip().lower()
31                 if 'temp:' in line and 'hum:' in line:
32                     parts = line.split(';')
33                     t = int(parts[0].replace('temp:', '').strip())
34                     h = int(parts[1].replace('hum:', '').strip())
35                     node6 = factory_state["stage_6_drying_chamber"]
36                     node6["temperature_celsius"] = t
37                     node6["humidity_percent"] = h
38                     node6["is_active"] = h > 30
39                     node6["fan_on"] = h > 40
40                     node6["sensor_connected"] = True
41                 except Exception as e:
42                     factory_state["stage_6_drying_chamber"]["sensor_connected"] =
False
43                     await asyncio.sleep(0.1)
44
45 async def forward_to_java_server():
46     while True:
47         try:
48             url = "http://localhost:8080/api/factory-status"
49             payload = json.dumps({"status": "ok", "data": factory_state}).
encode('utf-8')
50             req = urllib.request.Request(url, data=payload, headers={'
Content-Type': 'application/json'}, method='POST')
51             urllib.request.urlopen(req, timeout=1)
52         except Exception:
53             pass
54         await asyncio.sleep(1)

```

Листинг 12: Фрагмент hardware_bridge.py

5 Визуальный 2D-симулятор (Python / PyGame)

5.1 main.py — Графический SCADA-клиент OPC UA

Путь: pygame-simulator/main.py

Зависимости: PyGame, AsyncUA, AsyncIO, Aiohttp

Модуль реализует цифровой макет физического конвейера линии горячего цинкования. Включает графическую подсистему рендеринга на PyGame и асинхронный клиент OPC UA на базе библиотеки `asyncua`.

5.1.1 Двухъярусная топология линии

Комплекс смоделирован в виде двух связанных линий:

- **Линия 1 (Предварительная подготовка):** Станции 1–4 (Щелочное обезжиривание → Промывка 1 → Травление → Промывка 2);
- **Линия 2 (Нанесение покрытия):** Станции 5–7 (Флюсование → Сушильная камера с датчиком → Ванна горячего цинкования).

5.1.2 Автомат состояний мостового манипулятора

Каждая линия оснащена независимым мостовым краном, логика которого управляется конечным автоматом со следующими состояниями: IDLE → MOVE_TO_PICKUP → LIFT_DOWN_PICKUP → LIFT_UP_PICKUP → MOVE_TO_DROP → LIFT_DOWN_DROP → LIFT_UP_DROP.

Приоритеты диспетчеризации манипулятора: 1. Выгрузка готовой детали с финальной позиции на выходной конвейер (освобождение линии); 2. Транспортировка деталей между смежными ваннами от конца к началу; 3. Загрузка новой заготовки со входного накопителя на начальную позицию.

```
1 async def opcua_client_task():
2     url = "opc.tcp://localhost:4840/"
3     while True:
4         client = Client(url=url)
5         try:
6             await client.connect()
7             opc_data["connected"] = True
8             idx = await client.get_namespace_index('http://factory-aps.
9             local')
10
11             sched = await client.nodes.objects.get_child([f"{idx}:8
12             _Global_Scheduler"])
13             manual_node = await sched.get_child([f"{idx}:ManualOverride"])
14             status_node = await sched.get_child([f"{idx}:SystemStatus"])
15             step_node = await sched.get_child([f"{idx}:
16             CurrentAlgorithmStep"])
17             sim_node = await sched.get_child([f"{idx}:SimulationRunning
18             "])
19
20             # Подключение к 7 технологическим пакетам-узлам
21             f1 = await client.nodes.objects.get_child([f"{idx}:1
22             _Alkaline_Degreasing"])
```

```

18         s1_t = await f1.get_child([f"{idx}:Temperature"])
19         s1_a = await f1.get_child([f"{idx}:IsActive"])
20         s1_h = await f1.get_child([f"{idx}:HeaterOn"])
21
22         f6 = await client.nodes.objects.get_child([f"{idx}:6
_Drying_Chamber"])
23         s6_t = await f6.get_child([f"{idx}:TemperatureCelsius"])
24         s6_w = await f6.get_child([f"{idx}:HumidityPercent"])
25         s6_s = await f6.get_child([f"{idx}:SensorConnected"])
26
27         while True:
28             opc_data["manual_mode"] = await manual_node.get_value()
29             opc_data["status"] = await status_node.get_value()
30             opc_data["step"] = await step_node.get_value()
31             opc_data["sim_running"] = await sim_node.get_value()
32
33             opc_data["stage1"] = {"temp": await s1_t.get_value(), "
is_active": await s1_a.get_value(), "heater": await s1_h.get_value()}
34             opc_data["stage6"] = {"temp": await s6_t.get_value(), "hum
": await s6_w.get_value(), "sensor_connected": await s6_s.get_value()}
35             await asyncio.sleep(0.1)
36         except Exception:
37             opc_data["connected"] = False
38             await asyncio.sleep(2)

```

Листинг 13: Клиент AsyncUA для 8 узлов в pygame-simulator/main.py

6 Единый диспетчер запуска (C# / .NET / Release Launcher)

6.1 Launcher.cs — Оркестратор процессов и мониторинг здоровья

Путь: Release/Launcher.cs

Язык: C# (.NET Framework / Core)

Размер: 252 строки

Утилита обеспечивает скоординированный запуск всех компонентов системы в автономной промышленной сборке (standalone release).

6.1.1 Функции оркестратора

1. **Валидация окружения:** Проверка установленной виртуальной машины Java (`java -version`);
2. **Фоновый запуск ядра:** Старт `server.jar` с привязкой рабочего каталога;
3. **Поллинг готовности:** Циклическая проверка TCP-порта 8080 через `TcpClient` до подтверждения запуска REST API;
4. **Запуск UI:** Автоматическое открытие интерфейса дашборда в браузере по умолчанию и запуск графического симулятора `simulator.exe`;
5. **Перехват сигналов завершения:** Регистрация обработчика `Win32 API SetConsoleCtrlHandler` для гарантированного снятия процессов сервера и симулятора при закрытии консоли.

```
1 using System;
2 using System.Diagnostics;
3 using System.IO;
4 using System.Net.Sockets;
5 using System.Runtime.InteropServices;
6 using System.Threading;
7
8 class Program {
9     private static Process serverProcess = null;
10    private static Process simulatorProcess = null;
11
12    [DllImport("Kernel32")]
13    private static extern bool SetConsoleCtrlHandler(EventHandler handler,
14        bool add);
15    private delegate bool EventHandler(CtrlType sig);
16    private enum CtrlType { CTRL_C_EVENT = 0, CTRL_CLOSE_EVENT = 2 }
17
18    static void Cleanup() {
19        Console.WriteLine("\n[Launcher] Остановка всех сервисов...");
20        try { if (simulatorProcess != null && !simulatorProcess.HasExited)
21            simulatorProcess.Kill(); } catch { }
22        try { if (serverProcess != null && !serverProcess.HasExited)
23            serverProcess.Kill(); } catch { }
24    }
25 }
```

```

22
23     static bool CheckPort(int port) {
24         try {
25             using (TcpClient client = new TcpClient("127.0.0.1", port))
26             return true;
27         } catch { return false; }
28     }
29
30     static void Main(string[] args) {
31         SetConsoleCtrlHandler((sig) => { Cleanup(); return true; }, true);
32         AppDomain.CurrentDomain.ProcessExit += (s, e) => Cleanup();
33
34         string currentDir = AppDomain.CurrentDomain.BaseDirectory;
35         string jarPath = Path.Combine(currentDir, "server.jar");
36         string simPath = Path.Combine(currentDir, "simulator.exe");
37
38         // Запуск Java Backend
39         serverProcess = Process.Start(new ProcessStartInfo {
40             FileName = "java", Arguments = "-jar \"" + jarPath + "\"",
41             WorkingDirectory = currentDir, UseShellExecute = false
42         });
43
44         // Ожидание готовности порта 8080
45         int attempts = 0;
46         while (attempts++ < 25) {
47             if (CheckPort(8080)) break;
48             Thread.Sleep(1000);
49         }
50
51         Process.Start("http://localhost:8080");
52         if (File.Exists(simPath)) {
53             simulatorProcess = Process.Start(new ProcessStartInfo {
54                 FileName = simPath, WorkingDirectory = currentDir });
55         }
56     }

```

Листинг 14: Оркестрация и перехват сигналов завершения в Launcher.cs

7 Веб-дашборд (React / TypeScript / Vite)

7.1 App.tsx — SCADA-панель цифрового двойника

Путь: `src/App.tsx`

Стек: React 18, TypeScript, Vite, Tailwind CSS

Интерактивная панель мониторинга для инженера-технолога и оператора АСУ ТП. Осуществляет периодический опрос серверного ядра (каждые 500 мс по эндпоинтам `/api/aps-state` и `/api/queue`).

7.1.1 Функциональные блоки интерфейса

1. **Телеметрические карточки 7 стадий:** Визуализация статуса активности, температуры с цветовой градацией, состояния насосов, перемешивателей, ТЭНов и датчика DHT11;
2. **Интерактивная блок-схема APS:** Динамическая подсветка текущего шага алгоритма синим неоновым свечением, пройденных шагов — изумрудным цветом;
3. **Управление производственной очередью:** Отображение ожидающих партий с приоритетами, ручное добавление/удаление заказов;
4. **Пульт ручного вмешательства:** Активация режима `MANUAL_INTERVENTION`, кнопки принудительного вклинивания партии в начало расписания или постановки в конец.

```
1 useEffect(() => {
2   const fetchState = async () => {
3     try {
4       const [apsRes, queueRes] = await Promise.all([
5         fetch('http://localhost:8080/api/aps-state'),
6         fetch('http://localhost:8080/api/queue')
7       ]);
8       const apsData = await apsRes.json();
9       const queueData = await queueRes.json();
10      setApsState(apsData);
11      setQueueState(queueData);
12    } catch (e) {
13      console.error('Ошибка поллинга данных:', e);
14    }
15  };
16  const interval = setInterval(fetchState, 500);
17  return () => clearInterval(interval);
18 }, []);
```

Листинг 15: Фрагмент опроса и отображения стадий в `App.tsx`

8 Мобильный терминал (Android / Kotlin)

8.1 Архитектура мобильного клиента

Стек: Kotlin, Jetpack Compose, Retrofit 2, Moshi, StateFlow

Мобильное приложение представляет собой терминал оператора цеха, реализованный по архитектурному шаблону MVVM (Model-View-ViewModel).

- **ApiService.kt:** Описание REST-интерфейса Retrofit и DTO-моделей данных (SystemState, QueueState);
- **MainViewModel.kt:** Содержит реактивные потоки StateFlow и корутины viewModelScope, выполняющие периодический опрос сервера;
- **MainActivity.kt:** Декларативный пользовательский интерфейс на Jetpack Compose с Material Design 3.

```
1 package com.example.api
2
3 import retrofit2.http.GET
4 import retrofit2.http.POST
5 import retrofit2.http.Body
6
7 interface ApiService {
8     @GET("/api/state")
9     async fun getState(): SystemState
10
11     @GET("/api/queue")
12     async fun getQueue(): QueueState
13
14     @POST("/api/queue")
15     async fun addTask(@Body request: TaskRequest): TaskResponse
16
17     @POST("/api/toggle-manual")
18     async fun toggleManual(): SystemState
19 }
```

Листинг 16: Сетевой интерфейс Retrofit в ApiService.kt

9 Вспомогательные файлы и конфигурации

9.1 schedule.json — Персистентное расписание

Формат файла расписания на диске:

```
1 {  
2   "queue": [  
3     {  
4       "id": "BATCH_1721000000_001",  
5       "name": "Партия Сталь А-12",  
6       "material": "Сталь",  
7       "weight": 45.0,  
8       "priority": 78,  
9       "status": "WAITING",  
10      "steps": []  
11    }  
12  ],  
13  "current_tasks": []  
14 }
```

Листинг 17: Структура schedule.json

10 Заключение

Разработанный программно-аппаратный комплекс включает **18 программных модулей**, распределённых по 5 ключевым подсистемам: серверному ядру, аппаратному шлюзу HIL, графическому симулятору SCADA, оркестратору запуска и клиентским пользовательским интерфейсам. Общий объём разработанного исходного кода превышает **3500 строк**.

Ключевым достоинством архитектуры является её модульность, строгая типизация адресного пространства OPC UA и прямое взаимно-однозначное соответствие между теоретической блок-схемой алгоритма APS и сервисным слоем `ApsAlgorithmService.java`. Система обеспечивает бесперебойную синхронизацию физических данных датчиков с цифровым двойником и высокую гибкость при переходе в режим ручной диспетчеризации.